

Animation

Table des matières

Création de la mise en page	2
Icônes.	4
Utilisation des icones <i>Material Design</i>	5
Version finale de l'exemple	7
Exercices	12
Références.	12

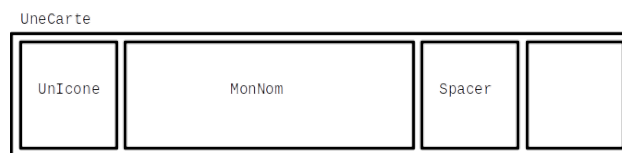
Dans ce chapitre, vous allez apprendre à ajouter une animation simple à votre application *Android*. Les animations permettent de rendre votre application plus interactive, plus intéressante et plus facile à interpréter. Animer des changements spécifiques sur un écran rempli d'informations peut aider l'utilisateur à identifier ce qui change.

Vous pouvez utiliser de nombreux types d'animations dans l'interface utilisateur d'une application. Les éléments peuvent apparaître ou disparaître en fondu, se déplacer sur l'écran ou en dehors, ou être transformés de façon intéressante. Cela rend l'interface utilisateur de l'application plus lisible et plus facile à utiliser.

Les animations permettent également d'améliorer l'aspect général de l'application, pour la rendre plus élégante tout en simplifiant son utilisation.

Création de la mise en page

Dans un premier temps, il est nécessaire de configurer un projet dans Android Studio avec un modèle *Empty Activity*. Nommez ce projet **Animation**. Puis, ajouter les 4 composants: **UneCarte()**, **UnIcône()**, **MonNom()** et **UnBouton()**.



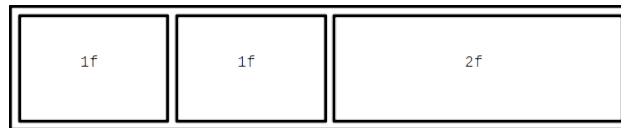
```

1 @Composable
2 fun UneCarte(modifier: Modifier = Modifier) {
3     Card(
4         modifier = modifier.padding(8.dp).fillMaxWidth(),
5         elevation = CardDefaults.cardElevation(
6             defaultElevation = 10.dp
7         ),
8         shape = RoundedCornerShape(0.dp)
9     ) {
10         Column(){
11             Row(){
12                 UnIcône()
13                 MonNom()
14                 Spacer(modifier.weight(1f))
15                 UnBouton()
16             }
17         }
18     }
19 }
20
21 @Composable
22 fun UnIcône(modifier: Modifier = Modifier) {
23     Text(
24         text = "Icône",
25         modifier = modifier
26     )
27 }
28
29 @Composable
30 fun MonNom(modifier: Modifier = Modifier) {
31     Column(modifier){
32         Text(
33             text = "Gill",
34             style = MaterialTheme.typography.displaySmall,
35             modifier = modifier
36         )
37         Text(
38             text = "Stephane",
39             style = MaterialTheme.typography.bodyLarge,
40             modifier = modifier
41         )
42     }
43 }
44
45 @Composable
46 fun UnBouton(modifier: Modifier = Modifier) {
47     Text(
48         text = "B",
49         modifier = modifier
50     )
51 }

```

À la ligne 14, la méthode `Modifier.weight()` définit la largeur et la hauteur de l'élément d'interface proportionnellement à sa pondération (en relation à celle de ses frères pondérés, qui sont les autres éléments enfants de la ligne ou de la colonne).

Par exemple, prenons trois éléments enfants sur une ligne, avec les pondérations `1f`, `1f` et `2f`. Dans ce cas, tous les éléments enfants ont une pondération. L'espace disponible pour la ligne est divisé proportionnellement à la valeur de pondération spécifiée. Davantage d'espace est réservé aux enfants dont les valeurs de pondération sont plus élevées. La pondération des éléments est représentée par la figure ci-dessous:



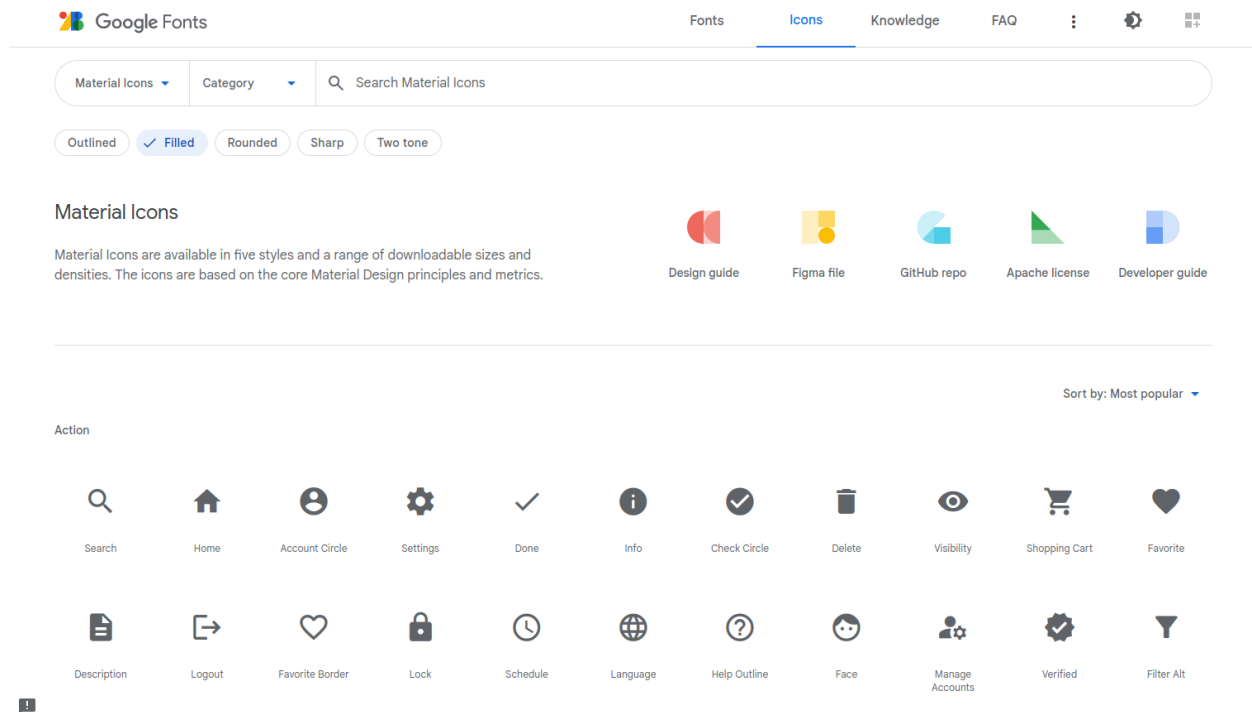
Le premier composant obtient un quart de la largeur, le deuxième un second quart, et le troisième obtient la moitié.

Par défaut, si les composants enfants n'ont pas de pondération (ce paramètre est facultatif), leur hauteur et leur largeur enveloppent le contenu (s'ajustant au contenu de l'élément d'UI).

Icônes

Les icônes sont des symboles qui aident les utilisateurs à comprendre l'interface utilisateur en communiquant visuellement la fonction souhaitée. Elles s'inspirent souvent d'objets du monde physique que l'utilisateur est supposé connaître. La conception des icônes réduit souvent le niveau de détail au minimum requis pour être reconnues par l'utilisateur. Par exemple, dans le monde physique, un crayon sert à écrire. Une icône de crayon symbolise généralement la **création** ou la **modification** d'un élément.

Material Design propose un certain nombre d'icônes classées dans des catégories courantes répondant à la plupart des besoins (<https://fonts.google.com/icons>).



Utilisation des icones *Material Design*

Pour utiliser les icones *Material Design* il est nécessaire d'ajouter la dépendance de bibliothèque **material-icons-extended** au projet.

1. Dans le volet **Project** (Projet), ouvrez **Gradle Scripts > build.gradle (Module: ...)**
2. Faire défiler la page jusqu'à la fin du fichier **build.gradle (Module: ...)**. Dans le bloc **dependencies{}**, ajoutez la ligne suivante :

```
1 implementation "androidx.compose.material:material-icons-extended"
```

3. *Android Studio* affiche un pop-up vous invitant à synchroniser votre projet. Cliquez sur **Sync Now** (Synchroniser).
4. Modifier le composant **MonIcone** pour y ajouter un icône de personnage.

```
1 @Composable
2 fun UnIcone(modifier: Modifier = Modifier) {
3     Icon(
4         imageVector = Icons.Filled.Face,
5         tint = MaterialTheme.colorScheme.secondary,
6         contentDescription = "face",
7         modifier = Modifier.size(72.dp).padding(4.dp)
8     )
9 }
```

5. Ajouter un bouton pour ouvrir ou fermer un volet déroulant.

```
1 @Composable
2 fun UnBouton(
3     ouvert: Boolean,
4     onClick: () -> Unit,
5     modifier: Modifier = Modifier
6 ) {
7     IconButton(
8         onClick = onClick,
9         modifier = modifier
10    ) {
11        Icon(
12            imageVector = if (ouvert) Icons.Filled.ExpandLess else Icons
13            .Filled.ExpandMore,
14            contentDescription = "Ouvrir",
15            tint = MaterialTheme.colorScheme.secondary
16        )
17    }
```

Le paramètre `onClick` (ligne 4) reçoit une fonction qui retourne rien (*CallBack*). Lorsque le bouton est pressé cette fonction est appelé. Tandis que le paramètre reçoit l'état de la carte. Ce qui permet de choisir l'icône à afficher (ligne 12).

6. Intégrer le bouton à la carte

```

1 @Composable
2 fun UneCarte(modifier: Modifier = Modifier) {
3     var ouvert = rememberSaveable { mutableStateOf(false) }
4     Card(
5         modifier = modifier.padding(8.dp).fillMaxWidth(),
6         elevation = CardDefaults.cardElevation(
7             defaultElevation = 10.dp
8         ),
9         shape = RoundedCornerShape(0.dp)
10    ) {
11        Column(
12            modifier = Modifier
13                .animateContentSize(
14                    animationSpec = spring(
15                        dampingRatio = Spring.DampingRatioNoBouncy,
16                        stiffness = Spring.StiffnessMedium
17                    )
18                )
19        ){
20            Row(){
21                UnIcone()
22                MonNom()
23                Spacer(modifier.weight(1f))
24                UnBouton(ouvert.value, {ouvert.value = !ouvert.value})
25            }
26            if(ouvert.value){
27                Text(
28                    text = "9155 rue Saint-Hubert",
29                    style = MaterialTheme.typography.bodyLarge,
30                    modifier = modifier
31                )
32                Text(
33                    text = "1 514 389-5921",
34                    style = MaterialTheme.typography.bodyLarge,
35                    modifier = modifier
36                )
37            }
38        }
39    }
40 }

```

À la ligne 3, une variable booléenne (**ouvert**) contenant l'état de la carte est défini. La gestion de l'état se fait par une fonction passer en paramètre au bouton (ligne 24). Les lignes 26 à 37 font en sorte que de l'information supplémentaire est affichée lorsque l'état **ouvert** est à **true**.

Version finale de l'exemple

```

1 package com.example.animation
2
3 import android.os.Bundle

```

```

4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.animation.animateContentSize
7 import androidx.compose.animation.core.Spring
8 import androidx.compose.animation.core.spring
9 import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.fillMaxWidth
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.shape.RoundedCornerShape
17 import androidx.compose.material.icons.Icons
18 import androidx.compose.material.icons.filled.ArrowDropDown
19 import androidx.compose.material.icons.filled.ExpandLess
20 import androidx.compose.material.icons.filled.ExpandMore
21 import androidx.compose.material.icons.filled.Face
22 import androidx.compose.material3.Card
23 import androidx.compose.material3.CardDefaults
24 import androidx.compose.material3.Icon
25 import androidx.compose.material3.IconButton
26 import androidx.compose.material3.MaterialTheme
27 import androidx.compose.material3.Surface
28 import androidx.compose.material3.Text
29 import androidx.compose.runtime.Composable
30 import androidx.compose.runtime.mutableStateOf
31 import androidx.compose.runtime.saveable.rememberSaveable
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.tooling.preview.Preview
34 import androidx.compose.ui.unit.dp
35 import com.example.animation.ui.theme.AnimationTheme
36
37 class MainActivity : ComponentActivity() {
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         setContent {
41             AnimationTheme {
42                 Surface(
43                     modifier = Modifier,
44                     color = MaterialTheme.colorScheme.background
45                 ) {
46                     UneCarte()
47                 }
48             }
49         }
50     }
51 }
52
53 @Composable
54 fun UneCarte(modifier: Modifier = Modifier) {
55     var ouvert = rememberSaveable { mutableStateOf(false) }
56     Card(
57         modifier = modifier.padding(8.dp).fillMaxWidth(),

```



```

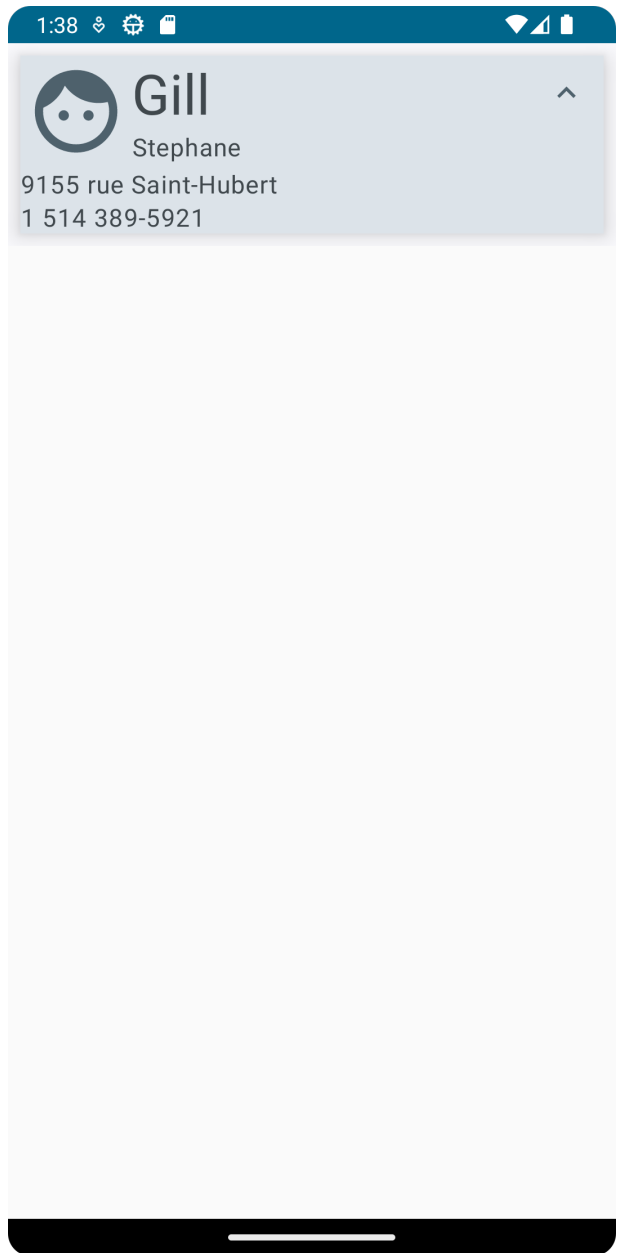
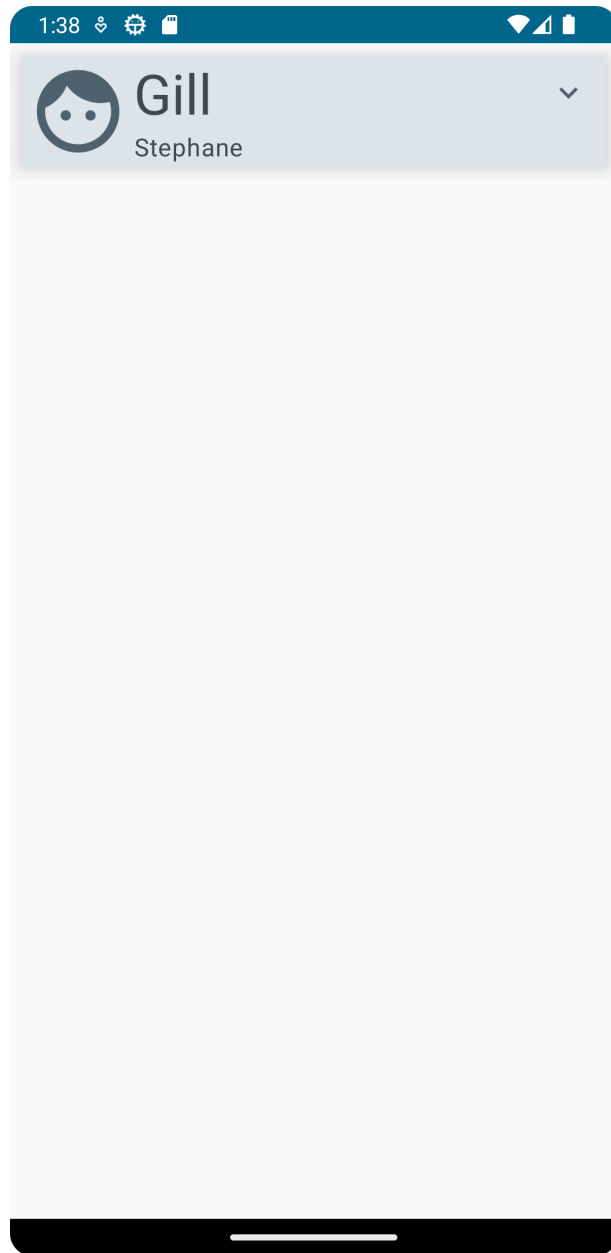
58         elevation = CardDefaults.cardElevation(
59             defaultElevation = 10.dp
60         ),
61         shape = RoundedCornerShape(0.dp)
62     ) {
63         Column(
64             modifier = Modifier
65                 .animateContentSize(
66                     animationSpec = spring(
67                         dampingRatio = Spring.DampingRatioNoBouncy,
68                         stiffness = Spring.StiffnessMedium
69                     )
70                 )
71         ){
72             Row(){
73                 UnIcone()
74                 MonNom()
75                 Spacer(Modifier.weight(1f))
76                 UnBouton(ouvert.value, {ouvert.value = !ouvert.value})
77             }
78             if(ouvert.value){
79                 Text(
80                     text = "9155 rue Saint-Hubert",
81                     style = MaterialTheme.typography.bodyLarge,
82                     modifier = modifier
83                 )
84                 Text(
85                     text = "1 514 389-5921",
86                     style = MaterialTheme.typography.bodyLarge,
87                     modifier = modifier
88                 )
89             }
90         }
91     }
92 }
93
94 @Composable
95 fun UnIcone(modifier: Modifier = Modifier) {
96     Icon(
97         imageVector = Icons.Filled.Face,
98         tint = MaterialTheme.colorScheme.secondary,
99         contentDescription = "face",
100         modifier = Modifier.size(72.dp).padding(4.dp)
101     )
102 }
103
104 @Composable
105 fun MonNom(modifier: Modifier = Modifier) {
106     Column(modifier){
107         Text(
108             text = "Gill",
109             style = MaterialTheme.typography.displaySmall,
110             modifier = modifier
111         )

```

```

112         Text(
113             text = "Stephane",
114             style = MaterialTheme.typography.bodyLarge,
115             modifier = modifier
116         )
117     }
118 }
119
120 @Composable
121 fun UnBouton(
122     ouvert: Boolean,
123     onClick: () -> Unit, // Fonction qui retourne rien (CallBack) Passer une
fonction en paramètre
124     modifier: Modifier = Modifier
125 ) {
126     IconButton(
127         onClick = onClick,
128         modifier = modifier
129     ) {
130         Icon(
131             imageVector = if (ouvert) Icons.Filled.ExpandLess else Icons
.Filled.ExpandMore,
132             contentDescription = "Ouvrir",
133             tint = MaterialTheme.colorScheme.secondary
134         )
135     }
136 }
137
138 @Preview(showBackground = true)
139 @Composable
140 fun GreetingPreview() {
141     AnimationTheme {
142         UneCarte()
143     }
144 }

```



Exercices

Exercice 1

Refaire l'exemple des notes de cours

Références

- [Animation simple avec Jetpack Compose](#)
- [Material Design Icon](#)